Food Store

Trabajo práctico 7 · NoSQL y nuevas formas de almacenamiento

Integrantes
Gonzalo Sanchez · Roman Avila · Mailen Ortiz
Leo Fachinelli · Maximiliano Mendez · Gabriel D'Anna

Repositorio de trabajo · Gonza-12-Sanchez/TPs_BD_FoodStore

1. Modelo documental

Se seleccionaron cinco pedidos y sus diez líneas del respaldo de prueba `foodstore_20261028_203928_224.dump` del TP6. El destino es la colección `food_store_tp7.pedidos`, con un documento por pedido.

Origen en PostgreSQL	Campo de destino en MongoDB
`pedido.id` (PK), fecha, estado, forma de pago y total	`_id`, `fecha`, `estado`, `forma_pago`, `total`
`pedido.usuario_id referencia usuario.id`	Objeto `cliente` con `id`
`detalle_pedido.id` (PK) y `pedido_id referencia pedido.id`	Arreglo `lineas` con `id`, `cantidad`, `precio_unitario` y `subtotal`
`detalle_pedido.producto_id referencia producto.id`	`lineas[].producto_id` y `lineas[].producto`

Los identificadores conservan su valor como cadenas. La fecha de origen es `DATE` y se conserva en formato ISO. Los importes se exportan como Extended JSON para cargarlos como BSON `Decimal128`.

2. Reglas de integridad y carga

Identidad y pertenencia. Cada pedido conserva su ID único, su cliente y las líneas asociadas. Cada línea conserva su propio ID y el del producto.

Cantidades e importes. Cada cantidad es un entero positivo. El precio unitario procede del detalle original, el subtotal equivale a cantidad por precio y el total es la suma de los subtotales.

Las líneas se embeben porque se consultan junto con el pedido y forman parte de su historial. Se guarda solo el identificador del cliente para asociar el pedido con su origen y evitar incluir datos personales.

El script de carga lee el archivo `pedidos_foodstore.ejson`, verifica documentos existentes e inserta los faltantes con `insertMany()`.

Consultas y verificación

Las consultas de MongoDB usan `find()` sobre el mismo lote, con proyección de `_id` y `estado`. La consulta por importe también muestra `total`.

Consulta MongoDB	Filtro adicional	IDs obtenidos
a. Todos los pedidos	Sin filtro adicional	1, 2, 3, 4, 5
b. Por estado	`estado: "PENDIENTE"`	1, 2
c. Por producto	`"lineas.producto_id": "4"`	2
d. Por importe	`total: { $gt: NumberDecimal("1000.00") }`	2, 4

El producto 4 es Pizza Pepperoni. El umbral de 1.000,00 separa los dos pedidos de mayor importe del lote, de 1.700,00 y 1.090,00.

Comparación entre motores

Consulta	PostgreSQL ejecutado	MongoDB ejecutado	Coincidencia en ejecución
Estado PENDIENTE	2 filas · IDs 1, 2	2 documentos · IDs 1, 2	Sí
Pizza Pepperoni	1 fila · ID 2	1 documento · ID 2	Sí

> **Evidencia disponible.** Las consultas SQL se ejecutaron en PostgreSQL 18.6 sobre una restauración del respaldo. El control de integridad devolvió cero filas. MongoDB cargó cinco documentos y ejecutó las cuatro consultas. La verificación automática de IDs terminó correctamente. Al repetir la carga, insertó cero documentos adicionales.

Documento de ejemplo · Pedido 2

Documento recuperado de MongoDB por el script y por Mongoose. Los importes se abrevian como texto en este extracto y usan `$numberDecimal` en la salida.


```
{
  "_id": "2", "fecha": "2026-10-28", "estado": "PENDIENTE",
  "forma_pago": "TARJETA", "total": "1700.00", "cliente": {"id": "2"},
  "lineas": [
    {"id": "3", "producto_id": "3", "producto": "Pizza Margherita",
     "cantidad": 1, "precio_unitario": "800.00", "subtotal": "800.00"},
    {"id": "4", "producto_id": "4", "producto": "Pizza Pepperoni",
     "cantidad": 1, "precio_unitario": "900.00", "subtotal": "900.00"}
  ]
}
```

Evaluación técnica

Consulta a ChatGPT

> Food Store usa PostgreSQL con tablas de pedidos, detalles, usuarios y productos vinculadas por claves foráneas. Consultamos con frecuencia pedidos completos y debemos conservar la coherencia de cantidades y precios históricos. ¿Qué ventajas y límites tendrían PostgreSQL y MongoDB para este caso?

Afirmación de ChatGPT	Contraste con documentación oficial
Embeber las líneas permite leer el pedido completo en una operación y actualizar campos del mismo documento de manera atómica.	MongoDB documenta ambas propiedades del embebido. La aplicación debe calcular o validar la relación entre subtotales y total. [1, 2]
Las claves foráneas de PostgreSQL mantienen la integridad referencial entre tablas.	PostgreSQL documenta que una clave foránea exige que los valores no nulos referencien una fila válida. El cálculo de importes requiere reglas adicionales. [3]

Conclusión

Conviene mantener PostgreSQL como fuente principal de Food Store. Las claves foráneas protegen las referencias a clientes y productos, y las transacciones permiten registrar de manera coherente el pedido y sus líneas. MongoDB ofrece una representación directa para consultar pedidos completos con sus precios históricos. Una colección secundaria podría servir para un flujo de lectura intensivo si las mediciones justifican mantener una copia y se define cómo sincronizarla. Las consultas por estado y producto devolvieron los mismos IDs en ambos motores. Antes de decidir una migración, conviene medir el rendimiento con datos representativos.

Complemento · Conexión con Mongoose

`mongoose_demo.js` abre la conexión con `mongoose.connect()` y define esquemas para pedido, cliente y líneas. El modelo valida campos requeridos, estados, cantidades e importes. Tras cargar el lote, la demostración lee y valida el pedido 2. La ejecución de `npm.cmd start` conectó con la base local, recuperó el pedido 2 y completó la validación sin errores. [4]

Referencias

[1] MongoDB · Embedded Data in Your MongoDB Schema
[2] MongoDB · Atomicity and Transactions
[3] PostgreSQL 18 · Foreign Keys
[4] Mongoose · Connections